

PHASE 1 — FOUNDATION LEARNING 📅 July 2025 – November 2025

Core Learning Topics

- 🌐 Networking Basics**
TCP/IP, DNS, Routing, Switching, Subnetting, VLANs
- 🔒 Security**
TLS/SSL, Firewalls, Auth, HSTS, Security Headers
- 🐳 AI Basics**
ML fundamentals, Optimization algos, Metaheuristics
- 🐳 Docker (Containerization)**
Images, Containers, Volumes, Networks, Compose
- 🐳 Proxmox (Virtualization)**
VMs, LXC containers, Cluster, Storage
- 🐳 Kubernetes (K8s)**
Pods, Deployments, Services, ReplicaSets, Namespaces, HPA, kubectl, kubednm

🔗 NGINX — Reverse Proxy + Load Balancer + Small Form App

Server Configuration:

- worker_processes auto
- worker_connections 1024
- sendfile on | log_notpush on | tcp_nodelay on
- keepalive_timeout 65
- server_tokens off (hide Nginx version)

TLS Security:

- Protocols: TLSv1.2 + TLSv1.3 only
- Ciphers: ECDHE-ECDSA-AES256-GCM-SHA384, ECDHE-RSA-CHACHA20-POLY1305, AES128-GCM...
- ssl_session_cache shared SSL:10m
- ssl_session_timeout 10m

Security Headers:

- HSTS: max-age=31536000; includeSubDomains
- X-Frame-Options: SAMEORIGIN
- X-Content-Type-Options: nosniff

Redirect: HTTP 80 → HTTPS 443 (301)

Upstream Pools (Round-Robin LB):

- upstream get_backend: → 127.0.0.1:3001 → 127.0.0.1:3002
- upstream modify_backend: → 127.0.0.1:6001 → 127.0.0.1:6002
- upstream frontend: → 127.0.0.1:5001 → 127.0.0.1:5002

Location Routes (HTTPS 443):

- /get → get_backend
- /modify → modify_backend
- / → frontend
- /assets/ → frontend (static)

Proxy Settings (all routes):

- proxy_http_version 1.1
- X-Real-IP + X-Forwarded-For headers
- proxy_connect_timeout 5s
- proxy_read_timeout 60s
- client_max_body_size 10M
- Logs: pfe_access.log / pfe_error.log

🔍 MONITORING STACK COMPARISON & SELECTION

- Zabbix**
 - Traditional monitoring
 - Agent-based collection
 - Complex configuration
 - Good SNMP support
 - Older architecture
 - Less cloud-native
- 🟢 PLG Stack (SELECTED)**
 - Prometheus (metrics scraping)
 - Loki (log aggregation)
 - Grafana (dashboards & UI)
 - Alertmanager (alert routing)
 - node_exporter on each VM
 - Cloud-native, lightweight
- ELK Stack**
 - Elasticsearch (storage/search)
 - Logstash (data pipeline)
 - Kibana (visualization)
 - Filebeat / agents
 - Heavy resource usage
 - Complex scaling

👉 Decision: PLG Stack chosen — lightweight, cloud-native, Prometheus-native alerting, integrates with AlterManager API

PHASE 1 — PRODUCTION TESTING 📅 December 2025

🏢 Production Application Ministry Microservices + Hospital Microservices

16 Physical Machines → Ubuntu Server installed on all 16
Test approach: deploy cluster → stress test → scale → improve → next cluster

🌐 CLUSTER 1 — Basic

- 1 Ministry node + 1 Hospital node
- No scaling, no redundancy
- Single node fails → entire service goes DOWN
- Purpose: Baseline prototype / testing environment

🌐 CLUSTER 2 — Scaled

- Ministry server: 2 replicas
- Hospital server: 3 replicas
- 1 replica fails → others continue serving requests
- Purpose: Handle more traffic + basic fault tolerance

🌐 CLUSTER 3 — Microservices

- Ministry: SoIn | Notifications
- Hospital: Frontend | Reception | Medicament
- Each service runs independently
- Purpose: Modularity, maintainability, independent scaling

🌐 CLUSTER 4 — Hybrid HA (High Availability)

- Microservices + Replicas (x2 per service):
- SoIn x2 | Notifications x2
- Frontend x2 | Reception x2 | Medicament x2

Container crash → another replica takes over automatically
Purpose: Production-grade HA + Fault Tolerance + Scalability

📊 Stress Tests + PLG Monitoring Results
Applied to all 4 clusters — measured: CPU usage, RAM, network throughput, HTTP response times, uptime, error rates, recovery time

🏆 RESULT: Hybrid HA (Cluster 4) = Best Architecture
Selected as the deployment strategy for Phase 2

✅ PHASE 1 COMPLETE

LEARNING DEEPDIVE 📅 January – February 2026

🔬 Advanced Study

- Proxmox Advanced
 - Cluster management
 - VM/LXC lifecycle & API
 - Storage: ZFS, LVM
 - SDN: Bridges, VLANs
 - API token zones
 - Backup & snapshots
 - HA Manager
 - proxmoxer Python lib
 - ProxmoxAPI client

- Kubernetes Advanced
 - kubernetes cluster setup
 - RBAC & namespaces
 - Persistent Volumes (PVC)
 - ConfigMaps / Secrets
 - kubectl & kubeconfig
 - HPA / replicas scaling
 - K8s Python client
 - Deployment patching API
 - Apps / Api / Core / V API

- Docker Advanced
 - Multi-stage builds
 - Docker networking modes
 - Registry & image mgmt
 - Resource limits
 - Docker in Proxmox VMs
 - Container security
 - docker-compose & stacks

PHASE 2 — INFRASTRUCTURE (Proxmox + K8s) 📅 February – March 2026 | Currently Ongoing

🖨️ Physical Hardware Setup

16 Physical PCs → Proxmox VE installed on all 16
→ Proxmox Cluster formed → 8 Physical Machines (PMs) joined as cluster nodes
(Remaining 8 kept as standby / test machines)

🔄 VM Deployment — Hybrid HA Approach

Initial VM Sizing: 3 GB RAM | 1 vCPU | 25 GB Disk
Ministry VMs (VLAN 2) → 10.20.20.0/24:
SoIn x2 | Notifications x2
Hospital VMs (VLAN 1) → 10.10.10.0/24:
Frontend x2 | Reception x2 | Medicament x2
Total: 10 app VMs + 1 Monitoring VM + 1 OPNsense VM

- Mini-Clusters Architecture (New Idea)
 - Divide full cluster into k mini-clusters
 - Optimizes provisioning algorithm: O(n) → O(n/k)
 - Organizes VM scaling and new VM creation
 - New cloned VM joins the same mini-cluster as its parent
 - Enables localized horizontal scaling decisions

- Monitoring VM (VLAN 3 → 10.30.30.0/24)
 - Prometheus — metrics scraping (node_exporter on every VM)
 - Loki — log aggregation
 - Grafana — dashboards & visualization
 - Alertmanager — sends webhooks to AlterManager API
 - blackbox_exporter — HTTP 5xx rate monitoring

🌐 VIRTUAL NETWORK ARCHITECTURE

Physical Network: 172.25.5.0/24
All 16 PMs + Router on this physical LAN
Physical switch is UNMANAGED (no VLAN tagging support)

🔧 OPNsense VM — Virtual Router/Firewall
Acts as vRouter bridging physical LAN (172.25.5.0/24) to virtual networks
Provides routing, firewall rules, and VLAN separation
Connected to all 3 VLANs + physical uplink

- VLAN 1 — Hospital**
10.10.10.0/24
Frontend, Reception, Medicament VMs
- VLAN 2 — Ministry**
10.20.20.0/24
SoIn, Notifications VMs
- VLAN 3 — Monitoring**
10.30.30.0/24
Prometheus, Grafana, Loki, Alertmanager

🔗 VXLAN SDN Zones (Proxmox) — Software-Defined Networking
Enables virtual L2 segments over unmanaged physical switch (L3 tunnel encapsulation)
Workaround for non-manageable physical switch

PHASE 2 — ALTERMANGER API 📅 Feb–Mar 2026

⚙️ System Flow

Prometheus Alertmanager → webhook → AlterManager API → Proxmox API (VMs/LXC) or Kubernetes API
Scheduler: cluster poll every 10s | warning queue process every 30s (max 1 instance each)

📄 Alert Label Schema (from Prometheus alerts.yml)

```
target_type : "vm" | "lxc" | "k8s"
severity : "warning" | "critical"
type : "cpu" | "memory" | "disk_io" | "network" | "http_5xx"
vmid : +15* (Proxmox VM/LXC ID)
namespace : "default" + deployment: "my-app" (K8s only)
```

🚨 Alert Thresholds & Auto-Scaling Rules

- CPU Usage**
 - 🔴 > 80% → WARNING → queue
 - 🔴 > 95% → CRITICAL → +1 vCPU (vertical)
- RAM Usage**
 - 🔴 > 60% → WARNING → queue
 - 🔴 > 80% → CRITICAL → +20% RAM (vertical)
- Disk IO (Free)**
 - 🔴 Free < 20% → WARNING → queue
 - 🔴 Free < 20% → CRITICAL → +15G disk (vertical)
- Network / HTTP Errors**
 - 🔴 HTTP 5xx > 1% → WARNING → queue
 - 🔴 HTTP 5xx > 5% → CRITICAL → clone VM (horizontal)
 - 🔴 Packet Loss > 2% → CRITICAL → clone VM (horizontal)

⚡ Scaling Decision Logic

- ⚠️ WARNING Path**
 - Add to warning_queue (one entry per VM, multi-alert)
 - Processed by DE-WOA on next 30s scheduler tick
 - Pre-emptive / proactive scaling
 - Multiple alert types merged per VM (CPU+RAM = 1 entry)
- 🔴 CRITICAL Path — Immediate Action**
 - CPU | MEMORY | DISK_IO → Vertical Scale now
 - NETWORK | HTTP_5XX → Horizontal Scale now
 - PM resources insufficient → Migrate VM (calls DE-WOA)
 - Energy cap exceeded → reject, log warning

Scaling Actions Detail:
Proxmox Vertical: +1 vCPU | +20% RAM (MB) | +15G disk (in-place config PUT, zero downtime for CPU/RAM)
Proxmox Horizontal: clone VM/LXC (full=1) → assign new VMID → start clone → joins parent mini-cluster
K8s Vertical: CPU +200 milicores | Memory +20% (patch deployment limits & requests)
K8s Horizontal: +1 replica (patch deployment scale)

📊 FITNESS FUNCTION — VM Placement Score

$$fitness = \sqrt{(w_1(\Delta C)^2 + w_2(\Delta R)^2 + w_3(\Delta IO)^2 + w_4(\Delta E)^2)}$$
$$\Delta X = \frac{resource_needed_by_VM - resource_available_on_node}{\Delta X - mean_AX} / \sigma_AX \rightarrow z\text{-score normalization}$$

Weights: W_CPU=0.35 | W_RAM=0.35 | W_IO=0.15 | W_E=0.15
Infeasible node (any $\Delta X > 0$) → returns infinity (rejected)
o source: rolling history window (500 samples) preferred; fallback to cross-sectional o during warm-up (<30 samples)
Energy model: 50W per CPU core | 10W per GB RAM
Real data from Prometheus preferred over Proxmox API estimates

🐳 HYBRID DE-WOA PROVISIONING ALGORITHM

Differential Evolution + Whale Optimization Algorithm (adapted)

- Randomly sample m PMs from n available online nodes
 - Map m VMs to m sampled PMs (random permutation)
 - Calculate total fitness = $\sum fitness_{vm_i, node_j}$
 - Remap 1 to 10 times → keep mapping with lowest fitness
 - Take a different random sample of m PMs
 - Repeat steps 1-4 with new sample
 - Repeat steps 5-6 for k=5 independent samples
 - Return mapping with best overall fitness across all k2 samples
- Container → VM assignment: Best Fit algorithm
→ Migration suggestion logged if better node found (manual action)

🔧 Prometheus Metrics Integration (Optional Enhancement)
4 real-time per-VM metrics fetched in parallel (ThreadPoolExecutor):
cpu_actual (cores in use) | ram_actual (bytes) | io_actual (bytes/s) | energy_actual (watts)
Queries configurable via env vars | Falls back to Proxmox API estimates if unavailable
Node IO capacity configurable: NODE_IO_CAPACITY_BPS (default 500 MB/s SATA SSD)

🔌 REST API Endpoints
POST /alertmanager/webhook — Receive Prometheus Alertmanager webhooks
GET /health — Health check + queue size + delta history stats
GET /nodes | /vms | /lxc | /deployments — Live cluster state
PUT /vms/vmid/resources — Manual vertical scale (VM)
PUT /lxc/vmid/resources — Manual vertical scale (LXC)
POST /vms/vmid/clone — Manual horizontal scale (VM)
POST /lxc/vmid/clone — Manual horizontal scale (LXC)
PATCH /deployments/{res}/{dep}/replicas — K8s manual replica set
GET /warning-queue — View pending DE-WOA items
DELETE /warning-queue/{id} — Remove item or specific alert type
GET /delta-history/stats — Fitness normalization diagnostics

PHASE 3 — AI INTEGRATION & AUTOMATION 📅 April – June 2026

🔬 Phase 2 Testing & Improvement

- Stress tests on Proxmox infrastructure
- Validate AlterManager scaling reactions
- Tune thresholds & DE-WOA parameters
- Validate fitness function accuracy
- Measure energy consumption
- Network performance benchmarks
- Compare downtime before vs. after
- Measure provisioning response time

🤖 AI Integration (Planned)

- Predictive scaling (ML time-series)
- Anomaly detection on metrics
- Workload forecasting models
- Intelligent threshold auto-tuning
- Reinforcement learning for provisioning
- Neural network for fitness estimation
- Auto-tuning DE-WOA k1, k2 params
- Load pattern recognition

🤖 Full Automation (Planned)

- Zero-touch VM lifecycle management
- Auto-remediation of failures
- Dynamic mini-cluster rebalancing
- Self-healing infrastructure
- Intelligent live migration decisions
- Automated capacity planning

🎓 FINAL PRESENTATION

June 2026
Bachelor Degree Defense
Smart Provisioning System — PFE